

Tablas hash, árboles y grafos

TC-CH05 – Técnicas Computacionales en Biología

Edición 2 · 2026-09-03

Pregunta del tema. Ninguna estructura lineal consigue a la vez búsqueda barata e inserción barata. ¿Qué hay que cambiar para romper ese compromiso, y qué se paga a cambio?

Producto final. Una tabla de dispersión de codones y un recorrido de grafo escritos por usted, y un veredicto razonado sobre qué nodo de una red de interacción es realmente crítico, sostenido por el cálculo y no por la intuición del grado.

Mapa del tema

1. El límite de lo lineal	1
2. Árboles y estructuras jerárquicas	2
3. Árboles Binarios de Búsqueda (BST)	3
4. Montículos binarios (<i>Heaps</i>) y colas de prioridad	5
5. Tablas Hash y resolución de colisiones	6
6. Grafos y redes biológicas	9
7. La operación peligrosa: borrado en direccionamiento abierto y ciclos	12
8. Actividad de depuración: errores en BST y grafos	12
9. Síntesis operativa y tablas de diagnóstico	13
10. Glosario conceptual	13
11. Conexión con el curso y siguientes pasos	13
12. Fuentes y reproducibilidad	14

Cómo usar este tema

El capítulo anterior terminó en un callejón: con estructuras lineales, o la búsqueda es barata o la inserción lo es, pero no las dos. Este capítulo rompe ese compromiso tres veces, de tres maneras distintas, y cada una cobra su precio. El árbol de búsqueda ordena y paga con el rebalanceo. La tabla de dispersión reparte y paga con las colisiones y con la pérdida del orden. El grafo abandona la jerarquía y paga con que casi todas sus preguntas cuestan más.

Al terminar sabrá: recorrer un árbol de búsqueda y explicar los tres casos del borrado, que es donde está la dificultad real; construir una función de dispersión y decir por qué la suya reparte bien o mal; elegir entre encadenamiento y direccionamiento abierto sabiendo qué falla en cada uno; recorrer un grafo en anchura y en profundidad; y, sobre todo, distinguir una métrica de red de una conclusión biológica.

La ruta **esencial** son el árbol de búsqueda con su borrado, la tabla de dispersión con su función y sus colisiones, y los dos recorridos de grafo. La ruta de **estudio** añade montículos, rotaciones, caminos mínimos, métricas de red y el anexo.

1 El límite de lo lineal

ESENCIAL En las estructuras de datos lineales del capítulo anterior, todo intento de diseño se enfrenta a un compromiso inevitable: en un array ordenado, la búsqueda binaria es rápida ($O(\log n)$), pero insertar un nuevo dato exige desplazar elementos ($O(n)$); en un array sin ordenar o una lista enlazada, la inserción al final o al inicio es instantánea ($O(1)$), pero buscar un elemento obliga a un escaneo exhaustivo ($O(n)$). Ninguna estructura lineal puede ofrecer búsqueda e inserción rápidas simultáneamente.

En las estructuras lineales, un array ordenado logra búsqueda $O(\log n)$ pero inserción $O(n)$, y un array sin ordenar logra lo contrario: no hay estructura lineal que logre ambas operaciones baratas a la vez.

Las estructuras no lineales y jerárquicas rompen radicalmente este límite:

- Los **árboles binarios de búsqueda balanceados** organizan los datos en jerarquías que permiten buscar, insertar y eliminar elementos en tiempo logarítmico ($O(\log n)$) simultáneamente.

- Las **tablas hash** utilizan funciones matemáticas de dispersión para lograr acceso directo en tiempo constante promedio ($O(1)$), independiente del volumen de datos.
- Los **grafos** generalizan todas las estructuras anteriores para modelar redes biológicas complejas donde lo fundamental son las relaciones e interacciones entre entidades moleculares.

2 Árboles y estructuras jerárquicas

Un **árbol** es una estructura de datos no lineal que organiza los elementos en una jerarquía de niveles conectados por relaciones de paternidad/filiación.

2.1 Terminología fundamental y anatomía de un árbol

- **Nodo**: cada elemento individual del árbol que contiene un valor y enlaces a otros nodos.
- **Raíz (Root)**: el único nodo superior del árbol que no posee ningún padre.
- **Hoja (Leaf)**: cualquier nodo terminal que no tiene ningún hijo.
- **Subárbol (Subtree)**: cualquier nodo del árbol junto con todos los descendientes que cuelgan de él (que constituye a su vez un árbol completo).
- **Arista (Edge)**: el enlace directo que une un nodo padre con su nodo hijo.
- **Nodo interno**: cualquier nodo que posee al menos un hijo (en árboles generales incluye la raíz no solitaria; en árboles con raíz se distingue habitualmente entre raíz, nodos internos intermedios y hojas).
- **Profundidad de un nodo**: longitud del camino en número de aristas desde la raíz hasta dicho nodo (la raíz tiene profundidad 0).
- **Altura de un árbol (h)**: longitud del camino más largo en número de aristas desde la raíz hasta la hoja más profunda (un árbol con un único nodo tiene altura $h = 0$ y 1 nivel de nodos; un árbol vacío se define convencionalmente con $h = -1$).

En un árbol, un nodo es cada elemento; la raíz es el nodo sin padre; una hoja es un nodo sin hijos; un subárbol es un nodo junto con todo lo que cuelga de él; y la altura es la longitud del camino más largo de la raíz a una hoja.

Atención. Contamos la altura en **aristas**, que es la convención habitual en análisis de algoritmos: un árbol de un solo nodo tiene altura 0. Otras fuentes, incluidas las diapositivas de la asignatura, cuentan nodos, con lo que sus valores salen una unidad mayores. Ninguna de las dos es incorrecta; lo que no se puede es mezclarlas dentro de un mismo cálculo, y en este capítulo se usa siempre la primera.

Una arista es la conexión entre un nodo padre y su hijo; un nodo interno es cualquier nodo que no es ni la raíz ni una hoja; la profundidad de un nodo es la longitud del camino en aristas desde la raíz hasta él (profundidad de la raíz: 0), mientras que la altura del árbol es la profundidad máxima de sus hojas. Cada nodo guarda un valor y tantos punteros como hijos tenga —dos en un árbol binario—, la misma idea de puntero de TC-CH04 extendida a más de una dirección por nodo.

2.2 Naturaleza recursiva de los árboles

Un árbol se define matemáticamente en términos de sí mismo: es una raíz de la cual cuelgan cero o más subárboles disjuntos. Esta naturaleza recursiva es la razón directa de que casi todos los algoritmos sobre árboles (inserción, búsqueda, recorridos) se implementen de forma natural mediante funciones recursivas con un caso base claro.

Un árbol se define recursivamente: un nodo raíz del que cuelgan cero o más subárboles, cada uno de ellos, a su vez, un árbol. Casi todo algoritmo sobre árboles se escribe de forma recursiva por esta razón.

2.3 Taxonomía de árboles binarios

En un **árbol binario**, cada nodo tiene como máximo dos hijos (denominados hijo izquierdo e hijo derecho). La clasificación formal distingue:

1. **Árbol Lleno (Full)**: cada nodo tiene estrictamente 0 o 2 hijos (nunca exactamente 1).

2. **Árbol Completo (Complete)**: todos los niveles están completamente llenos, excepto posiblemente el último, que se llena de forma compacta de izquierda a derecha. Esta propiedad permite almacenarlo en un array contiguo sin punteros, indexando los hijos del nodo i en $2i + 1$ y $2i + 2$.
3. **Árbol Perfecto (Perfect)**: es un árbol lleno donde todas las hojas se encuentran exactamente al mismo nivel de profundidad.
4. **Árbol Degenerado o Sesgado (Skewed)**: cada nodo padre tiene un único hijo, comportándose de forma idéntica a una lista enlazada.
5. **Árbol Balanceado**: la diferencia de altura entre el subárbol izquierdo y derecho de cualquier nodo está acotada.

Un árbol binario lleno tiene en cada nodo 0 o 2 hijos, nunca exactamente 1. Un árbol completo tiene todos los niveles llenos salvo, como mucho, el último, relleno de izquierda a derecha —forma que permite almacenarlo en un array indexando los hijos del nodo i en $2i + 1$ y $2i + 2$ —. Un árbol perfecto es un árbol lleno donde además todas las hojas están al mismo nivel. Un árbol degenerado o sesgado tiene un único hijo por nodo padre y se comporta como una lista enlazada. Ninguna de estas cuatro propiedades equivale a estar balanceado, aunque un árbol perfecto siempre lo está.

Un árbol perfecto de altura h contiene exactamente:

$$N = \sum_{k=0}^h 2^k = 2^{h+1} - 1 \text{ nodos}$$

Para una altura $h = 3$, el árbol tiene $2^4 - 1 = 15$ nodos.

Un árbol perfecto de altura h tiene exactamente $2^{h+1} - 1$ nodos: cada nivel k (de 0 a h) está completamente lleno con 2^k nodos, y $\sum_{k=0}^h 2^k = 2^{h+1} - 1$. Para $h = 3$, un árbol perfecto tiene $2^4 - 1 = 15$ nodos.

Se reproduce con `verification/chapter_results.sh perfect_tree_nodes 3`.

2.4 El poder del balanceo: la altura logarítmica

En un árbol balanceado de n nodos, cada nivel duplica la capacidad del anterior (1, 2, 4, 8, ...). Para almacenar n elementos solo se necesitan:

$$h \approx \log_2(n) \text{ niveles}$$

Dado que las operaciones de búsqueda y recorrido descienden por un único camino desde la raíz hasta una hoja, el número máximo de pasos es proporcional a la altura: **coste** $O(\log n)$.

En un árbol binario balanceado de n nodos, la altura es del orden de $\log_2(n)$: duplicar el número de nodos solo añade un nivel. Como las operaciones cuestan proporcionalmente a la altura, son $O(\log n)$.

3 Árboles Binarios de Búsqueda (BST)

Un **Árbol Binario de Búsqueda (BST)** impone una invariante de orden estricta: para cada nodo con clave k , todos los valores en su subárbol izquierdo son menores ($< k$) y todos los valores en su subárbol derecho son mayores ($> k$).

Un árbol binario de búsqueda mantiene, para todo nodo, los valores del subárbol izquierdo menores que él y los del derecho mayores; con esta invariante, buscar, insertar y borrar sobre un árbol balanceado cuestan $O(\log n)$.

```
ALGORITMO: BST_Insercion_y_Busqueda
ESTRUCTURA DE NODO: {clave, valor, izq: puntero, der: puntero}
INVARIANTE BST: Para todo nodo u: todo x en subarbol_izq(u) tiene x.clave < u.clave; todo y en subarbol_der(u) tiene y.clave > u.clave
COMPLEJIDAD: Tiempo  $O(h)$  donde  $h$  es la altura del árbol ( $O(\log n)$  en árbol balanceado,  $O(n)$  en peor caso degenerado)

FUNCION BST_Buscar(raiz, clave_buscada):
    Si raiz == NULL entonces:
        Retornar NO_ENCONTRADO
    Si clave_buscada == raiz.clave entonces:
        Retornar raiz.valor
```

```

Sino si clave_buscada < raiz.clave entonces:
    Retornar BST_Buscar(raiz.izq, clave_buscada)
Sino:
    Retornar BST_Buscar(raiz.der, clave_buscada)

FUNCION BST_Insertar(raiz, nueva_clave, nuevo_valor):
    Si raiz == NULL entonces:
        Retornar Crear_Nuevo_Nodo(nueva_clave, nuevo_valor)

    Si nueva_clave < raiz.clave entonces:
        raiz.izq <- BST_Insertar(raiz.izq, nueva_clave, nuevo_valor)
    Sino si nueva_clave > raiz.clave entonces:
        raiz.der <- BST_Insertar(raiz.der, nueva_clave, nuevo_valor)
    Sino:
        raiz.valor <- nuevo_valor    # Actualizar valor asociado a clave existente

    Retornar raiz

```

3.1 Los tres casos del borrado en un BST

Eliminar un nodo en un BST requiere preservar la invariante de orden:

1. **Nodo hoja (sin hijos):** se desvincula directamente ($O(1)$ tras localizarlo).
2. **Nodo con un único hijo:** el padre del nodo a eliminar enlaza directamente con su nieto.
3. **Nodo con dos hijos:** se localiza su **sucesor in-order** (el nodo con menor valor en su subárbol derecho, hallado descendiendo hacia la izquierda). Se copia el valor del sucesor en el nodo a borrar y luego se elimina el sucesor original de su posición (que por construcción tiene a lo sumo un hijo derecho).

Para borrar un nodo de un BST: si es una hoja, se elimina directamente; si tiene un solo hijo, se sustituye por ese hijo; si tiene dos hijos, se busca su sucesor in-order (el valor más pequeño de su subárbol derecho, hallado descendiendo lo más a la izquierda posible), se copia su valor en el nodo a borrar, y se elimina el nodo sucesor de su posición original —que por construcción tiene como mucho un hijo derecho, cayendo siempre en uno de los dos casos anteriores—.

Sobre el árbol de referencia (raíz 5, hijos 3 y 8, hojas 1, 4, 7, 9), borrar la raíz —dos hijos— exige el sucesor in-order: el valor más pequeño del subárbol derecho (raíz 8) se halla descendiendo a la izquierda desde 8, que llega directamente a 7 (hoja, sin hijo izquierdo). Se sustituye 5 por 7 y se desengancha la hoja 7 original; el árbol resultante tiene raíz 7, in-order 1,3,4,7,8,9 y pre-order 7,3,1,4,8,9 —el árbol original con el 5 eliminado y la invariante de orden intacta—.

Se reproduce con `verification/chapter_results.sh bst_delete_two_children`.

3.2 El riesgo de degeneración y árboles autobalanceados

Si se insertan datos que ya vienen ordenados (10, 20, 30, 40, 50), un BST ingenuo inserta cada elemento sucesivamente en la rama derecha. El árbol **degenera en una lista enlazada de altura n** , degradando la búsqueda a $O(n)$.

Insertar 10, 20, 30, 40 y 50 en ese orden en un árbol de búsqueda sin rebalanceo produce un árbol degenerado de altura 4: una cadena de cinco nodos con cuatro aristas desde la raíz hasta la última hoja, frente a la altura 2 de un árbol balanceado con esos mismos cinco valores. Buscar el último valor insertado cuesta entonces $O(n)$ y no $O(\log n)$.

Se reproduce con `verification/chapter_results.sh bst_degenerate 5`.

Para garantizar la cota $O(\log n)$ en el peor caso se utilizan árboles autobalanceados mediante **rotaciones**:

Concepto. Antes de ver los tipos, conviene entender por qué una rotación está permitida. Una rotación cambia qué nodo es el padre y cuál el hijo, pero deja intacto el recorrido in-order del árbol, es decir, la secuencia ordenada de sus claves. Esa es exactamente la razón de que sea una operación válida sobre un árbol de búsqueda: la invariante de orden se conserva, mientras que la altura, que era el problema, se reduce. Rebalancear no reordena

los datos, solo los recoloca.

- **Árboles AVL:** exigen que las alturas de los subárboles de cada nodo difieran como máximo en 1 ($|\text{balance}| \leq 1$). Imponen un balance estricto que reduce la altura media a costa de requerir rotaciones frecuentes en inserciones y borrados.
- **Árboles Rojo-Negro:** relajan ligeramente la exigencia mediante etiquetas de color (ningún nodo rojo tiene hijo rojo y todo camino raíz-hoja tiene el mismo número de nodos negros). Requieren menos rotaciones en escenarios con mutaciones frecuentes.

Criterio de diseño: Ambos garantizan altura $O(\log n)$; la elección no depende de una superioridad universal, sino del balance entre consultas y actualizaciones en la carga de trabajo real.

Un árbol AVL exige que las alturas de los dos subárboles de cada nodo difieran como mucho en 1 (balance casi perfecto, búsquedas muy rápidas, rotaciones frecuentes en inserción/borrado); un árbol rojo-negro relaja esa exigencia mediante un etiquetado de color (ningún nodo rojo tiene hijos rojos; todo camino raíz-hoja atraviesa el mismo número de nodos negros), suficiente para $O(\log n)$ sin balance perfecto, con menos rotaciones—inserción/borrado más baratos— a cambio de búsquedas algo más lentas que un AVL. La elección sigue el patrón de uso: AVL cuando las búsquedas dominan ampliamente, rojo-negro cuando inserciones y borrados son frecuentes.

3.3 Recorridos sobre árboles

1. **In-order (Izquierda → Raíz → Derecha):** en un BST produce los elementos en estricto orden ascendente ($O(n)$).
2. **Pre-order (Raíz → Izquierda → Derecha):** procesa la raíz antes que sus descendientes; ideal para clonar o serializar árboles.
3. **Post-order (Izquierda → Derecha → Raíz):** procesa los hijos antes que el padre; ideal para liberar memoria y evaluar expresiones.

El recorrido in-order (izquierda, raíz, derecha) de un BST devuelve sus elementos en orden ascendente; el pre-order (raíz, izquierda, derecha) sirve para copiar o serializar la estructura, no para ordenar; y el post-order (izquierda, derecha, raíz) es el orden para evaluar expresiones y para borrar el árbol sin dejar nodos huérfanos.

Para el BST con raíz 5, hijos 3 y 8, y hojas 1, 4, 7, 9, el pre-order es 5, 3, 1, 4, 8, 7, 9 (no ordenado) y el in-order es 1, 3, 4, 5, 7, 8, 9 (ascendente): confundir un pre-order con «la lista ordenada» es un error de tipo de recorrido, no de implementación.

Se reproduce con `verification/chapter_results.sh bst_traversals`.

4 Montículos binarios (*Heaps*) y colas de prioridad

Un **montículo binario** (*Heap*) es un árbol binario completo que cumple la propiedad de heap: en un **Max-Heap**, la clave de cada nodo es mayor o igual que la de sus hijos (el máximo absoluto reside siempre en la raíz).

Un heap es un árbol binario completo donde cada padre es mayor o igual que sus hijos (max-heap); consultar el máximo cuesta $O(1)$ —siempre está en la raíz—, e insertar o extraer el máximo cuesta $O(\log n)$ porque el elemento se reordena a lo largo de la altura, logarítmica, del árbol. Es la implementación eficiente de la cola de prioridad que TC-CH04 dejó pendiente.

Un heap se almacena de forma extraordinariamente compacta en un array contiguo sin necesidad de punteros:

- Hijo izquierdo de i : $2i + 1$.
- Hijo derecho de i : $2i + 2$.
- Padre de i : $\lfloor (i - 1)/2 \rfloor$.

```
heapInsertar(heap, valor):  
    heap.append(valor)
```

```

i <- heap.length - 1
mientras i > 0 y heap[padre(i)] < heap[i]:
    intercambiar(heap[i], heap[padre(i)])
    i <- padre(i)                # Burbujeo ascendente (sift-up)

heapExtraerMax(heap):
    maximo <- heap[0]
    heap[0] <- heap.pop_last()
    i <- 0
    mientras heap[i] tenga un hijo mayor que heap[i]:
        j <- indice_del_hijo_mayor(i)
        intercambiar(heap[i], heap[j])
        i <- j                  # Hundimiento descendente (sift-down)
    devolver maximo

```

Insertando 5, 3, 8, 1, 9 en ese orden en un max-heap representado como array, el heap final es [9, 8, 5, 1, 3] —9 y 8 burbujearon hasta cerca de la raíz, 3 y 1 no necesitaron moverse—. Extraer el máximo de ese heap devuelve 9 y deja [8, 3, 5, 1]: el último elemento (3) ocupa la raíz vacía y se hunde una posición, hasta el nivel en que ya no tiene un hijo mayor que él.

Se reproduce con `verification/chapter_results.sh heap_insert_trace.`

y `heap_extract_max.`

5 Tablas Hash y resolución de colisiones

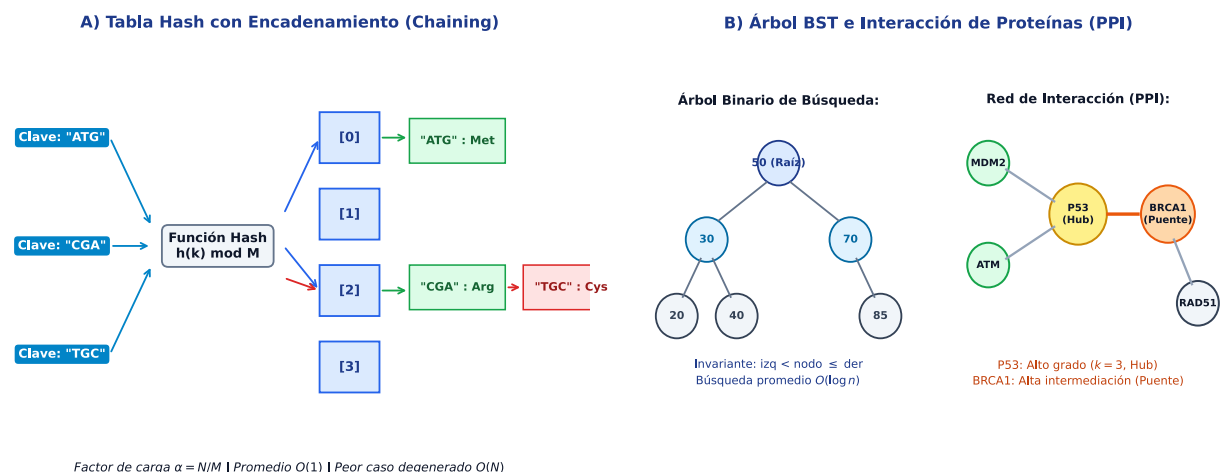


Figura 1: A) Tabla Hash con función de dispersión $h(k)$, factor de carga $\alpha = N/M$ y resolución de colisiones por encadenamiento (*chaining*). B) Árbol Binario de Búsqueda (BST) con invariante de orden $izq < nodo \leq der$, y Red de Interacción de Proteínas (PPI) diferenciando nodos de alto grado (*hubs*) frente a nodos puente (*bottlenecks* con alta intermediación). Figura original adaptada de las presentaciones docentes.

Una **tabla hash** es una estructura asociativa que mapea pares `clave` $\rightarrow$ `valor` mediante una función matemática de hash $h(\text{clave})$ que transforma directamente una clave en un índice entero dentro del rango $[0, M - 1]$ de la tabla, logrando un acceso directo en tiempo constante promedio $O(1)$.

Una tabla hash usa una función de hash que transforma una clave directamente en un índice de una tabla, evitando recorrer la colección; la búsqueda por clave cuesta $O(1)$ de media.

5.1 De qué está hecha una función de dispersión

ESENCIAL Hasta aquí la función $h(k)$ ha sido una caja negra. Abrirla es sencillo y explica por qué las tablas de dispersión funcionan. Una función útil cumple tres condiciones:

- **Determinista:** la misma clave produce siempre el mismo índice. Sin esto no se podría recuperar nada de lo guardado, que es lo primero que uno esperaría de un almacén.
- **Uniforme:** reparte las claves por todo el rango disponible sin preferir unas casillas sobre otras. Es la condición que mantiene cortas las cadenas de colisión y, con ellas, el coste medio constante.
- **Barata:** calcularla debe costar mucho menos que recorrer la colección. Si no, no compensa.

La familia más común en la práctica es la *dispersión universal*:

$$h_{a,b}(k) = (ak + b) \bmod p \bmod M$$

donde p es un número primo mayor que cualquier clave posible, M es el tamaño de la tabla, y a y b se eligen al azar con $1 \leq a < p$ y $0 \leq b < p$. Elegir a y b al azar es lo que impide que un conjunto concreto de claves degrade la tabla de forma sistemática: ninguna distribución particular de datos, ni nadie que quisiera provocarlo a propósito, puede prepararse contra una función que no conoce de antemano.

Atención. Tomar simplemente el resto de la división, $h(k) = k \bmod M$, funciona en los ejemplos de clase y es lo que usamos aquí por claridad. Pero con M par y claves mayoritariamente pares, esa función llena solo la mitad de la tabla y duplica la longitud de todas las cadenas. La uniformidad no viene de regalo: es una propiedad que hay que buscar, y comprobar.

5.2 Colisiones y Factor de Carga (α)

Una **colisión** ocurre cuando dos claves distintas generan el mismo índice ($h(k_1) = h(k_2)$). El rendimiento de la tabla depende del **factor de carga**:

$$\alpha = \frac{N}{M} = \frac{\text{número de elementos insertados}}{\text{capacidad de la tabla}}$$

El $O(1)$ de una tabla hash es un promedio, no una garantía: depende de que la función de hash reparta bien las claves y de que el factor de carga (elementos entre tamaño de tabla) se mantenga bajo. Con una función que reparte mal o una tabla muy llena, la búsqueda degrada hacia $O(n)$ por el alargamiento de las cadenas de colisión.

Se reproduce con `verification/chapter_results.sh hash_load_factor 10 7`.

Cuando el factor de carga supera el umbral $\alpha > 0.7$, la tabla se redimensiona al doble ($2M$) y se ejecuta un **rehash completo** de todas las claves existentes.

Una tabla hash de tamaño 5 con encadenamiento, tras insertar 3 claves, tiene factor de carga $3/5=0.6$ (60%), por debajo del umbral 0.7, y no se redimensiona; al insertar una cuarta clave el factor de carga sube a $4/5=0.8$ (80%), supera el umbral y dispara un redimensionamiento a tamaño 10 seguido de un rehash completo de las 4 claves ya insertadas.

Se reproduce con `verification/chapter_results.sh hash_resize`.

5.3 Encadenamiento frente a Direccionamiento Abierto

- **Encadenamiento separado (*Chaining*):** cada cubeta almacena una pequeña lista enlazada con los elementos colisionados.
- **Direccionamiento abierto (*Open Addressing*):** todos los elementos residen en la propia tabla. Ante una colisión, se sondea secuencialmente:
 - *Sondeo lineal:* prueba índices consecutivos $+1, +2, \dots$ (sufre agrupamiento primario o *clustering*).
 - *Sondeo cuadrático:* incrementa el paso con potencias cuadráticas $+1^2, +2^2, +3^2, \dots$
 - *Doble Hashing:* calcula el salto mediante una segunda función de hash $h_2(k)$.

El direccionamiento abierto resuelve colisiones sondeando otras posiciones de la propia tabla en vez de encadenar: el sondeo lineal prueba el siguiente índice $(+1, +2, \dots)$, el cuadrático incrementa cuadráticamente $(+1, +4, +9, \dots)$, y el doble hashing calcula el incremento con una segunda función de hash. El sondeo lineal tiene

el mejor comportamiento de caché pero sufre agrupamiento primario (las colisiones se acumulan en bloques consecutivos, aumentando la probabilidad de nuevas colisiones para claves no relacionadas); el doble hashing evita el agrupamiento a costa de más cálculo y peor caché; el cuadrático queda entre ambos. A diferencia del encadenamiento, no necesita memoria extra para listas, pero satura antes y su borrado es más delicado porque vaciar una casilla sin más rompería la cadena de sondeo de claves posteriores.

Insertando las claves 12, 3, 17 y 8, en ese orden, en una tabla de tamaño 5 con sondeo lineal y hash $\text{clave} \% 5$: 12 y 3 caen directamente en sus índices naturales (2 y 3, sin sondeo); 17 colisiona en el índice 2 (ocupado por 12), sondea 3 (ocupado por 3) y se coloca en 4 tras 2 sondeos; 8 colisiona en el índice 3 (ocupado por 3), sondea 4 (ocupado ahora por 17) y, al llegar al final de la tabla, da la vuelta (*wrap-around*) hasta el índice 0, libre, tras 2 sondeos. La clave 8, que no colisiona directamente con 12 ni con 17 en su hash original, termina desplazada por el efecto en cadena del agrupamiento: una colisión entre otras dos claves puede desplazar a una tercera que no compartía su índice natural con ninguna de ellas, algo que no ocurre con encadenamiento, donde cada cubeta es independiente de las demás.

Se reproduce con `verification/chapter_results.sh open_addressing_linear`.

5.4 Caso bioinformático: Traducción de codones de ARNm

El código genético se modela de forma natural como una tabla hash donde la clave es un triplete nucleotídico (codón) y el valor es el aminoácido correspondiente.

Para traducir la secuencia AUGAAUCUGCCU:

1. Con marco 0 (inicio 0, paso 3): se leen los codones no solapados AUG (Met), AAU (Asn), CUG (Leu) y CCU (Pro).
2. Con marco 1 (inicio 1, paso 3): empezando en la posición 1 y avanzando de tres en tres salen UGA, que es un **codón de parada prematuro**, AUC (isoleucina) y UGC (cisteína).
3. Con marco 2 (inicio 2, paso 3) la lectura vuelve a ser otra: GAA, UCU y GCC.

Tres marcos, tres proteínas incompatibles a partir de la misma secuencia de nucleótidos. Un desplazamiento de marco (*frameshift*) causado por una inserción o un borrado de una sola base no cambia una parte del mensaje: cambia el mensaje entero a partir de ese punto, y por eso suele ser mucho más grave que una sustitución.

Aclaración metodológica: Avanzar con paso 1 desde el origen generaría todos los tripletes solapados (k -mers con $k = 3$: AUG, UGA, GAA, ...), operación computacional útil para indexado genómico pero conceptualmente distinta de un marco de lectura biológico.

Sobre AUGAAUCUGCCU, el marco 0 da AUG-AAU-CUG-CCU, es decir metionina, asparagina, leucina y prolina; el marco 1 da UGA-AUC-UGC, una parada prematura seguida de isoleucina y cisteína; y el marco 2 da GAA-UCU-GCC, glutámico, serina y alanina.

Se reproduce con `verification/chapter_results.sh reading_frames AUGAAUCUGCCU`.

Se reproduce con `verification/chapter_results.sh translate_codons AUGAAUCUGCCU 3`.

Leer codones con un paso de 1 en vez de 3 produce lecturas solapadas y fuera de marco; sobre AUGAAUCUGCCU el segundo codón leído sería UGA, un codón de parada real, en vez del AAU (Asparagina) que da la lectura correcta con paso 3.

Se reproduce con `verification/chapter_results.sh translate_codons AUGAAUCUGCCU 1`.

Operación	Lista sin ordenar	Array ordenado	BST balanceado	Tabla Hash
Búsqueda	$O(n)$	$O(\log n)$	$O(\log n)$	$O(1)$ promedio
Inserción	$O(1)$	$O(n)$	$O(\log n)$	$O(1)$ promedio
Borrado	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$ promedio
Recorrido ordenado	No soportado	$O(n)$	$O(n)$ (in-order)	No soportado

Si solo se necesita buscar/insertar por clave, la tabla hash es la mejor opción; pero si además hace falta recorrer

los datos en orden o consultar rangos, la tabla hash no sirve —no guarda orden— y el árbol balanceado es la elección, porque combina $O(\log n)$ con recorrido in-order ordenado.

6 Grafos y redes biológicas

Un **grafo** $G = (V, E)$ es la estructura más general en computación, formada por un conjunto de vértices o nodos V y un conjunto de aristas E que conectan pares de vértices. Modela interacciones físicas proteína-proteína (PPI), redes de regulación génica, rutas metabólicas y grafos de ensamblaje genómico (de Bruijn).

Un grafo modela entidades (nodos) y sus relaciones (aristas); las listas y los árboles son casos particulares de grafo. Se usa cuando lo esencial del problema son las relaciones entre entidades, no una secuencia ni una jerarquía.

Un grafo $G=(V,E)$ puede ser dirigido o no dirigido según si sus aristas tienen un solo sentido, y ponderado si sus aristas llevan un peso numérico; es disperso si $|E|$ es mucho menor que $|V|^2$ y denso si se acerca a ese máximo. Las redes biológicas suelen ser dispersas.

6.1 Representación: Matriz de adyacencia frente a Lista de adyacencia

- **Matriz de adyacencia** ($|V| \times |V|$): tabla bidimensional donde $M[u][v]$ almacena la existencia o peso de la arista. Ofrece comprobación de adyacencia instantánea en tiempo constante $O(1)$, pero requiere memoria cuadrática $O(|V|^2)$ independientemente de las aristas reales. Es la estructura ideal para **grafos densos** (por ejemplo, una matriz de correlación completa entre 200 muestras clínicas, donde la matriz es compacta y se consultan pares continuamente).
- **Lista de adyacencia**: cada nodo mantiene un vector con sus vecinos directos. Consume memoria proporcional a los datos reales $O(|V| + |E|)$, siendo la estructura óptima para **redes biológicas dispersas** (como redes de interacción proteína-proteína con millones de nodos y pocas conexiones por nodo, donde una matriz reservaría un volumen prohibitivo de memoria prácticamente vacía).

Una red dispersa de un millón de nodos representada con matriz de adyacencia ocuparía del orden de 10^{12} celdas, independientemente de cuántas aristas tenga realmente; representada con lista de adyacencia ocupa $O(V + E)$, proporcional a las aristas reales: para redes dispersas, la matriz es inviable en memoria y la lista es la elección correcta.

Se reproduce con `verification/chapter_results.sh matrix_vs_list_memory 1000 5000`.

6.2 Algoritmos de recorrido: BFS y DFS

- **Búsqueda en Anchura (BFS — *Breadth-First Search*)**: explora el grafo concéntricamente nivel a nivel empleando una **cola FIFO**. Calcula el camino más corto en número de aristas. Coste: $O(V + E)$.
- **Búsqueda en Profundidad (DFS — *Depth-First Search*)**: se adentra por una rama hasta el fondo antes de retroceder, empleando una **pila LIFO** o recursión. Detecta ciclos y componentes conexas. Coste: $O(V + E)$.

```
BFS(grafo, origen):
    visitados <- {origen}
    cola <- [origen]
    orden <- []
    mientras cola no este vacia:
        nodo <- cola.pop_front()
        orden.append(nodo)
        para cada vecino en grafo.vecinos(nodo):
            si vecino no esta en visitados:
                visitados.add(vecino)      # Marcar al encolar para evitar duplicados
                cola.append(vecino)
    devolver orden
```

BFS recorre un grafo por niveles usando una cola FIFO; DFS se adentra por una rama antes de retroceder, usando una pila o recursión. Sobre el mismo grafo, el orden de visita de BFS y de DFS es distinto; ambos cuestan $O(V + E)$.

6.3 Caminos mínimos en grafos ponderados

- **Algoritmo de Dijkstra:** calcula el camino de mínimo coste en grafos con pesos **no negativos** utilizando una cola de prioridad (Heap) para expandir siempre el nodo con menor distancia acumulada ($O((V + E) \log V)$).
- **Algoritmo de Bellman-Ford:** soporta aristas con pesos negativos relajando todas las aristas $V - 1$ veces ($O(V \cdot E)$).

BFS encuentra el camino con menos aristas, que solo coincide con el de menor coste si el grafo no tiene pesos (o todos valen lo mismo); con pesos, un camino de más aristas puede tener menor coste total, y BFS no lo detecta. El algoritmo adecuado con pesos no negativos es Dijkstra, que usa una cola de prioridad para expandir siempre el nodo de menor distancia acumulada.

Dijkstra solo funciona correctamente con pesos no negativos: su estrategia da por cerrado un nodo en cuanto lo expande, lo que puede dar resultados incorrectos si hay aristas de peso negativo. Para grafos con pesos negativos se usa Bellman-Ford, que relaja las aristas $V - 1$ veces.

6.4 Métricas de centralidad y análisis topológico

1. **Grado (k):** número de conexiones directas de un nodo. En redes dirigidas se desglosa en **grado de entrada** (*in-degree*) y **grado de salida** (*out-degree*). En redes de regulación génica, un gen con muchas salidas y pocas entradas es candidato a **factor de transcripción maestro**.
2. **Centralidad de intermediación** (*Betweenness Centrality*): proporción de caminos más cortos de toda la red que transitan obligatoriamente por un nodo dado.
3. **Centralidad de cercanía** (*Closeness Centrality*): inverso de la suma de distancias geodésicas desde el nodo hasta todos los demás.

ESTUDIO Las tres anteriores describen un nodo. Otras tres describen la red entera:

- **Grado medio**, que mide la densidad: cuántas interacciones tiene por término medio cada proteína.
- **Distancia media y diámetro.** El diámetro es la mayor de todas las distancias mínimas entre pares de nodos. Un diámetro corto indica que cualquier perturbación se propaga en pocos pasos, lo que importa al razonar sobre el efecto de un fármaco.
- **Coficiente de agrupamiento**, la proporción de vecinos de un nodo que además son vecinos entre sí. Un valor alto sugiere módulos funcionales, complejos que actúan juntos; uno bajo sugiere una red más parecida a una aleatoria.

Atención. Las tres son descripciones, no explicaciones, y vale para ellas lo mismo que para las anteriores: describen la topología observada, que depende tanto de la biología como de qué interacciones se han medido y cuáles no. Una proteína muy estudiada acumula aristas porque se ha buscado en ella, no necesariamente porque sea más central.

El grado de un nodo cuenta sus conexiones directas; la centralidad de intermediación cuenta la proporción de caminos más cortos que pasan por él; la centralidad de cercanía es el inverso de la suma de distancias a todos los demás nodos. Estas tres métricas pueden señalar nodos distintos como «el más importante» según qué relación midan.

```
ALGORITMO: Calculo_Centralidad_Grado_Red_Proteica
ENTRADA:
- Grafo_PPI: Lista de adyacencia de la red de interaccion proteica con V proteinas y E interacciones
SALIDA:
- Vector_Grados: Mapeo de cada proteina u a su grado deg(u)
- Lista_Hubs: Proteinas cuyo grado supera el umbral k_hub (ej. media + 2 desviaciones estandar)
COMPLEJIDAD: Tiempo  $O(V + E)$  sobre listas de adyacencia, Espacio auxiliar  $O(V)$ 
```

```

1. INICIALIZACION:
   Para cada proteina v en V:
       Grado[v] <- 0

2. CONTEO DE ARISTAS:
   Para cada proteina u en V:
       Para cada vecino w en Grafo_PPI.Adyacentes(u):
           Grado[u] <- Grado[u] + 1   # Cuenta de interacciones directas

3. DETECCION DE PROTEINAS HUB:
   Grado_Medio <- Suma(Grado) / |V|
   Lista_Hubs <- Conjunto_Vacio()

   Para cada proteina u en V:
       Si Grado[u] >= 2 * Grado_Medio entonces:
           Agregar (u, Grado[u]) a Lista_Hubs

4. RETORNO:
   Retornar (Grado, Lista_Hubs)

```

En un grafo dirigido, el grado se desdobra en grado de entrada (aristas que llegan al nodo) y grado de salida (aristas que salen de él). En una red de regulación génica dirigida, el grado de salida de un gen mide a cuántos otros regula y el grado de entrada mide cuántos reguladores lo controlan; un gen con grado de salida alto y grado de entrada bajo es candidato a regulador maestro, no a objetivo regulado, una distinción que el grado no dirigido no puede establecer.

6.5 El experimento del nodo puente

Considere una red compuesta por dos módulos triangulares ($\{1,2,3\}$ y $\{5,6,7\}$) unidos exclusivamente a través del nodo 4:

```

  1 --- 2
   \   /
    3 (Grado 3, distancias = 11)
    |
    4 (Grado 2, puente critico, distancias = 10)
    |
    5 (Grado 3, distancias = 11)
   /   \
  6 --- 7

```

Figura 2: Estructura de dos comunidades triangulares unidas por el nodo puente 4. El nodo 4 tiene menor grado pero mayor centralidad de intermediación y cercanía. Figura original.

En un grafo de dos triángulos unidos solo por el nodo 4, eliminar el nodo 3 (grado 3, el más alto) separa la red en componentes de tamaño 2 y 4; eliminar el nodo 4 (grado 2, menor) la separa en dos componentes iguales de tamaño 3 y 3. Ambas eliminaciones fragmentan la red, pero la de mayor impacto estructural —partirla en dos mitades— es la del nodo de menor grado: el grado por sí solo no identifica qué nodo sostiene la conectividad entre las dos comunidades.

Se reproduce con `verification/chapter_results.sh bridge_node`.

En el grafo de dos triángulos unidos por el nodo 4, la suma de distancias del nodo 4 al resto de los seis nodos es 10 (distancias 2,2,1,1,2,2); la del nodo 3 es 11 (distancias 1,1,1,2,3,3). El nodo 4, pese a tener menor grado (2 frente a 3), tiene mayor centralidad de cercanía —menor suma de distancias— porque es el único paso obligado hacia el triángulo opuesto: grado y cercanía pueden coincidir en señalar al mismo nodo como importante, o no, según qué relación midan.

Se reproduce con `verification/chapter_results.sh closeness centrality`.

6.6 Cautela biológica: las métricas son hipótesis

Identificar una proteína con alta centralidad (*hub* o *bottleneck*) en una red de interacción sugiere que podría ser biológicamente esencial, pero **no constituye una prueba definitiva**. Requiere siempre validación experimental en el laboratorio húmedo (*knockout*, ensayos funcionales).

Que un nodo tenga alta centralidad en una red biológica lo hace candidato a ser funcionalmente importante, pero no lo demuestra: confirmar que una proteína es esencial exige evidencia biológica (experimentos, anotación funcional), no solo la topología de la red. Una lista de «nodos más centrales» no es, por sí sola, una lista de “elementos biológicamente más importantes”.

7 La operación peligrosa: borrado en direccionamiento abierto y ciclos

Dos errores críticos en estructuras no lineales:

1. **Borrado ingenuo en direccionamiento abierto:** eliminar una clave dejando la celda como «vacía» rompe la cadena de sondeo de claves posteriores, haciéndolas ilocalizables. Se debe marcar la celda con un centinela especial (*tombstone* o «borrado»).
2. **Recorrido de grafos sin marcado de visitados:** en grafos con ciclos, omitir el conjunto de visitados provoca bucles infinitos y desbordamiento de pila.

Protocolo preventivo en 3 pasos:

1. Marcar siempre los nodos como visitados en el momento exacto de encolarlos en BFS o apilarlos en DFS.
2. Utilizar centinelas de borrado en tablas hash con direccionamiento abierto.
3. Monitorizar el factor de carga α antes de cada inserción masiva.

8 Actividad de depuración: errores en BST y grafos

Analice el siguiente código con tres errores conceptuales:

```
buscarGrafo(grafo, origen, destino):
    cola <- [origen]                # Fallo 1
    mientras cola no este vacia:
        actual <- cola.pop_front()
        si actual == destino: devolver True
        para cada vecino en grafo[actual]: # Fallo 2
            cola.append(vecino)          # Fallo 3
    devolver False
```

Tarea. Localice los tres fallos y clasifíquelos: cuál impide que el algoritmo termine, cuál lo rompe con un error visible y cuál devuelve un resultado incorrecto en silencio.

Diagnóstico.

1. No se comprueba que *origen* exista en el grafo. Si no está, la primera vuelta intenta consultar sus vecinos y falla, o peor, devuelve falso como si el destino fuera inalcanzable cuando en realidad la pregunta estaba mal planteada.
2. Se accede a `grafo[actual]` sin contemplar un nodo sin vecinos registrados. En muchos lenguajes eso lanza un error; en otros devuelve algo vacío y el fallo pasa desapercibido, que es lo peor de las dos opciones.
3. Se encola sin llevar un conjunto de visitados. Con un ciclo en el grafo, y las redes biológicas están llenas de ciclos, el bucle no termina nunca: los mismos nodos entran una y otra vez en la cola.

Concepto. El tercero es el que define el algoritmo. Un recorrido en anchura sin conjunto de visitados no es un recorrido en anchura lento: es un programa que no termina. La estructura auxiliar no es un detalle de eficiencia, es parte de la corrección.

9 Síntesis operativa y tablas de diagnóstico

9.1 Tabla de diagnóstico de síntomas en estructuras no lineales

Síntoma observado	Causa probable	Comprobación y corrección
Búsqueda en BST tarda $O(n)$ en vez de $O(\log n)$	Datos insertados en orden (árbol degenerado)	Rebalancear mediante AVL o Árbol Rojo-Negro
Búsqueda en tabla hash se ralentiza	Factor de carga $\alpha > 0.7$ o mala función de hash	Duplicar capacidad y ejecutar rehash completo
Clave existente no se encuentra tras borrado	Rotura de cadena en direccionamiento abierto	Usar centinelas de borrado (<i>tombstones</i>)
Recorrido en grafo no termina (bucle infinito)	Falta de marcado de visitados ante ciclos	Registrar <code>visitados.add(nodo)</code> al encolar

9.2 Tabla de síntesis con preguntas de control

Requisito del sistema	Estructura recomendada	Pregunta de control
Búsqueda exacta por clave $O(1)$	Tabla Hash	¿El factor de carga se mantiene < 0.7 ?
Búsqueda rápida y recorrido ordenado	Árbol Binario de Búsqueda (AVL/Rojo-Negro)	¿Se prevé inserción de datos ordenados?
Acceso continuo al elemento prioritario	Montículo binario (<i>Heap</i>)	¿Se necesita un Max-Heap o un Min-Heap?
Modelado de interacciones moleculares	Grafo con listas de adyacencia	¿La red es dirigida o ponderada?

10 Glosario conceptual

- **BST (Binary Search Tree):** árbol con invariante $izq < k < der$.
- **Árbol AVL:** BST autobalanceado con diferencia de alturas ≤ 1 .
- **Heap (Montículo):** árbol completo con propiedad de orden padre/hijo.
- **Tabla Hash:** estructura asociativa directa mediante función de dispersión.
- **Factor de carga (α):** cociente N/M entre elementos y capacidad.
- **Encadenamiento:** resolución de colisiones mediante listas por cubeta.
- **Direccionamiento abierto:** resolución de colisiones mediante sondeo interno.
- **BFS (Breadth-First):** recorrido por niveles con cola FIFO.
- **DFS (Depth-First):** recorrido en profundidad con pila LIFO.
- **Betweenness:** centralidad por proporción de caminos más cortos.

11 Conexión con el curso y siguientes pasos

Distinción conceptual: Árbol Binario de Búsqueda frente a Árbol Filogenético.

Aunque un BST y un árbol filogenético comparten algoritmos generales de recorrido jerárquico, sus principios matemáticos y significados biológicos son completamente distintos:

- En un **BST**, la ramificación izquierda/derecha responde a un orden total de claves alfanuméricas ($izq < k < der$) para optimizar la búsqueda de datos.

- En un **árbol filogenético**, la topología representa una hipótesis de parentesco y divergencia evolutiva; las ramas cuantifican distancias o tiempos biológicos y los nodos internos modelan ancestros comunes hipotéticos, sin ninguna relación de orden de búsqueda.

Reutilizar una técnica de recorrido no autoriza a interpretar un árbol filogenético como una estructura indexada de búsqueda.

Continuidad con los temas posteriores.

Las estructuras no lineales de este capítulo sustentan la ordenación eficiente con montículos en TC-CH06, la cuantificación de información en TC-CH07, los modelos probabilísticos en TC-CH08, el alineamiento sobre grafos dirigidos acíclicos en TC-CH09 y el aprendizaje profundo en TC-CH10.

- En **TC-CH06**, utilizaremos montículos (*heaps*) para implementar el algoritmo de ordenación Heapsort y compararemos estructuras contiguas y enlazadas.
- En **TC-CH07** y **TC-CH08**, las tablas hash y matrices estructurarán el conteo de frecuencias, la entropía y los modelos ocultos de Markov (HMMs).
- En **TC-CH09**, el alineamiento de secuencias se formulará como la búsqueda del camino óptimo en grafos dirigidos acíclicos (DAGs) mediante programación dinámica.
- En **TC-CH10**, organizaremos representaciones matriciales y grafos computacionales para redes neuronales artificiales.

12 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente de estructuras de datos de la asignatura, escrita por Álvaro Serrano Navarro. Los árboles de búsqueda, los montículos, la dispersión universal y los recorridos de grafo siguen a Cormen y colaboradores [3]; la traducción de codones y el uso de tablas para representar el código genético, a Compeau y Pevzner [2]; y el análisis de redes de interacción, a Allesina y Wilmes [1].

Todos los resultados se reproducen con `verification/chapter_results.sh` tal como están impresos, con sus argumentos.

Bibliografía

- [1] Stefano Allesina and Madlen Wilmes. *Computing Skills for Biologists: A Toolbox*. Princeton University Press, 2019. Ejemplos contrastivos de árboles y grafos; no sostiene ninguna afirmación en solitario.
- [2] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. 3rd edition, 2018. Encuadre bioinformático de la tabla hash de codones y de la interpretación de redes de interacción.
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009. Referencia canónica para árboles autobalanceados (AVL, rojo-negro), tablas hash y algoritmos de grafos (Dijkstra, Bellman-Ford, Floyd-Warshall); no se reproduce, solo se cita.